# Usage guide for ctags

# What is ctags?

- **Unix command that makes database of programming sources's tag (ex. Global variable, function definition, macro declaration)**

- **We can recognize where function and variable are using tags file.**

# Generate tags file

- **Command** `ctags -R *` on pintos/src directory.
  - Then ctags create tags file.

```
[redacted]@cspro9:~/pintos/src$ ctags -R *
[redacted]@cspro9:~/pintos/src$ ls -al
total 232
drwxr-xr-x 12 [redacted]  4096 Oct  5 00:46 .
drwxr-xr-x  3 [redacted]  4096 Aug 28  2011 ..
-rw-r--r--  1 [redacted]  1868 Oct  5 00:46 cscope.files
drwxr-xr-x  2 [redacted]  4096 Sep  8 04:32 devices
drwxr-xr-x  3 [redacted]  4096 Dec 28  2009 examples
drwxr-xr-x  2 [redacted]  4096 Sep  8 04:29 filesys
-rw-r--r--  1 [redacted]    34 Dec 28  2009 .gitignore
drwxr-xr-x  4 [redacted]  4096 Dec 28  2009 lib
-rw-r--r--  1 [redacted]  4621 Dec 28  2009 LICENSE
-rw-r--r--  1 [redacted]  1664 Dec 28  2009 Make.config
-rw-r--r--  1 [redacted]   628 Dec 28  2009 Makefile
-rw-r--r--  1 [redacted]  3810 Dec 28  2009 Makefile.build
-rw-r--r--  1 [redacted]   333 Dec 28  2009 Makefile.kernel
-rw-r--r--  1 [redacted]  1551 Dec 28  2009 Makefile.userprog
drwxr-xr-x  2 [redacted]  4096 Dec 28  2009 misc
-rw-r--r--  1 [redacted]151552 Oct  5 00:48 tags
drwxr-xr-x  8 [redacted]  4096 Dec 28  2009 tests
drwxr-xr-x  3 [redacted]  4096 Sep 21 02:32 threads
drwxr-xr-x  2 [redacted]  4096 Sep  8 04:29 userprog
drwxr-xr-x  2 [redacted]  4096 Sep 19 00:26 utils
drwxr-xr-x  2 [redacted]  4096 Sep  8 04:29 vm
```

# Register tags file at .vimrc

- **Add line at .vimrc file**

```
71 set tags=./tags;
72
~/.vimrc [+]
```

- **Then when launch vim, it finds tags file.**

# Shortcuts when using ctags on vim

- **Ctrl + ]**
  - Go to declaration point of function or variable.

- **Ctrl + t**
  - Go back previous tags or code.

- **Google it to see more commands or shortcuts.**

# Example

```c
/* Starts preemptive thread scheduling by enabling interrupts.
   Also creates the idle thread. */
void
thread_start (void)
{
  /* Create the idle thread. */
  struct semaphore start_idle;
  sema_init (&start_idle, 0);
  thread_create ("idle", PRI_MIN, idle, &start_idle);

  /* Start preemptive thread scheduling. */
  intr_enable ();

  /* Wait for the idle thread to initialize idle_thread. */
  sema_down (&start_idle);
}
```

**Ctrl + ]**

```c
tid_t
thread_create (const char *name, int priority,
               thread_func *function, void *aux)
{
  struct thread *t;
  struct kernel_thread_frame *kf;
  struct switch_entry_frame *ef;
  struct switch_threads_frame *sf;
  tid_t tid;
  enum intr_level old_level;

  ASSERT (function != NULL);

  /* Allocate thread. */
  t = palloc_get_page (PAL_ZERO);
  if (t == NULL)
    return TID_ERROR;

  /* Initialize thread. */
  init_thread (t, name, priority);
  tid = t->tid = allocate_tid ();
```

# Example

```c
tid_t
thread_create (const char *name, int priority,
               thread_func *function, void *aux)
{
  struct thread *t;
  struct kernel_thread_frame *kf;
  struct switch_entry_frame *ef;
  struct switch_threads_frame *sf;
  tid_t tid;
  enum intr_level old_level;

  ASSERT (function != NULL);

  /* Allocate thread. */
  t = palloc_get_page (PAL_ZERO);
  if (t == NULL)
    return TID_ERROR;

  /* Initialize thread. */
  init_thread (t, name, priority);
  tid = t->tid = allocate_tid ();
```

**Ctrl + t**

```c
/* Starts preemptive thread scheduling by enabling interrupts.
   Also creates the idle thread. */
void
thread_start (void)
{
  /* Create the idle thread. */
  struct semaphore start_idle;
  sema_init (&start_idle, 0);
  thread_create ("idle", PRI_MIN, idle, &start_idle);

  /* Start preemptive thread scheduling. */
  intr_enable ();

  /* Wait for the idle thread to initialize idle_thread. */
  sema_down (&start_idle);
}
```